Electronic Signatures

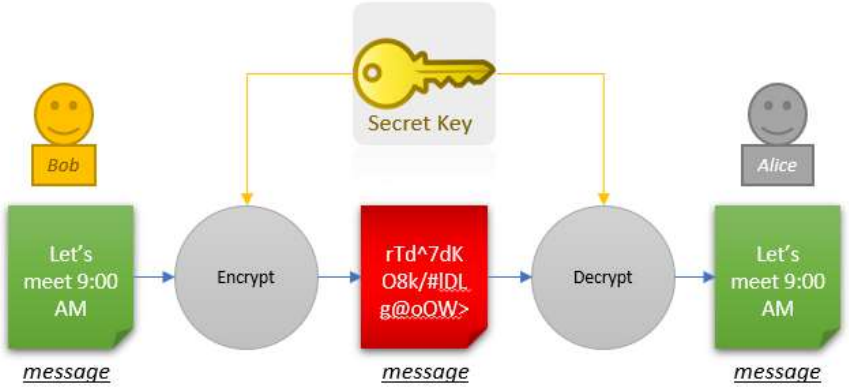
Mihai TOGAN
mihai.togan@mta.ro

Agenda

- Symmetric and asymmetric cryptography – short overview
- Intro to digital signatures
- Signing with RSA
- Signing with DSA
- EC Cryptography (ECC)
- ECDSA
- Digital Signature Profiles
  - CMS (PKCS#7) profile
  - ETSI CAdES profile
  - ETSI CAdES baseline profiles

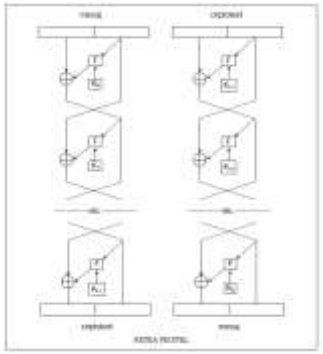
Symmetric key Cryptography

- Data encryption (confidentiality)
- Use the same key to encrypt and decrypt
- Computationally fast



- Data Encryption Standard (DES)
  - Block Cipher, 56 bit key / 64 bit block size / 16 rounds
  - Based on Feistel network – same circuit used to encrypt & decrypt
  - Triple DES 112 bit key (2-keys) or 168 bit key (3-keys)

$$c = E_{K3}\left(D_{K2}\left(E_{K1}(m)\right)\right), \qquad m = D_{K1}\left(E_{K2}\left(D_{K3}(c)\right)\right)$$



- Advanced Encryption Standard (AES)
  - Block Cipher 128, 192, 256 bit keys / 128 bit block size / 10, 12, 14 rounds
  - Rijndael Algorithm
  - Belgian cryptographers, Joan Daemen and Vincent Rijmen
  - Four transformations: *Substitution, ShiftRows, MixColumns, AddRoundKey*

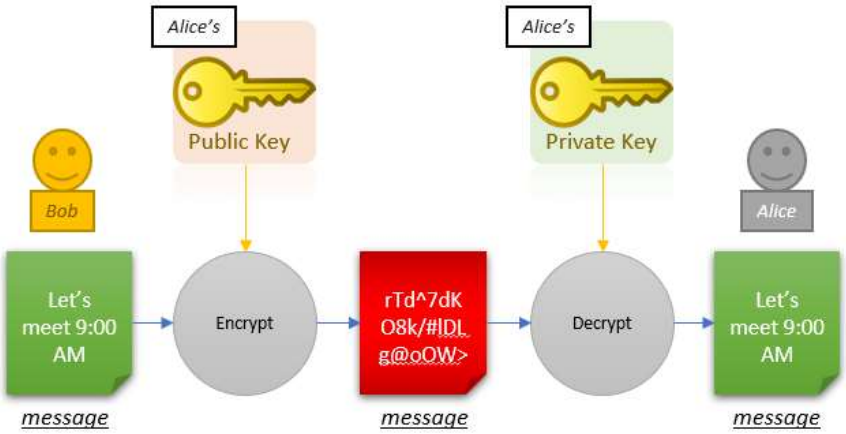
Asymmetric key Cryptography

- Data encryption
- Key agreement
- Data signing

- Use of two mathematically related keys
- Unable to derive one from the other
  - One public key published for all to use
  - Other is private key kept secret by owner

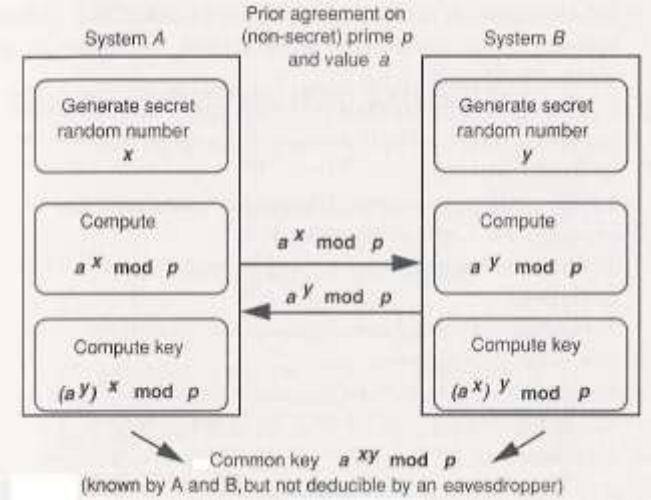
- Computationally slow

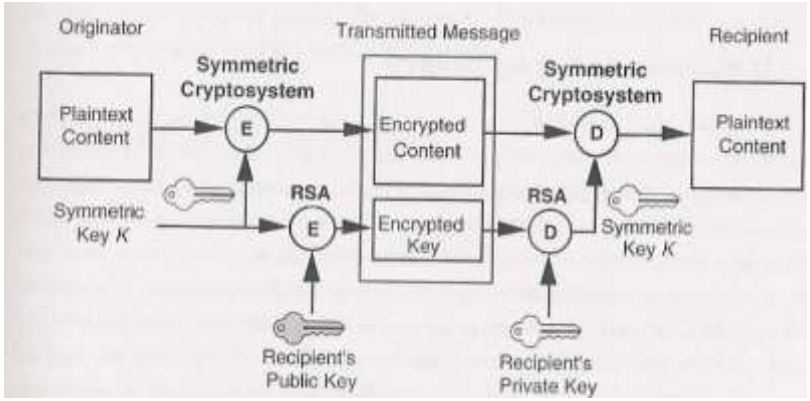
- Based on some hard problems
  - RSA - Integer Factorization (large primes)
  - Diffie-Hellman - Discrete Logarithms
  - ECC - Elliptic Curve Discrete Logarithm



Key Management aspects

- Key management is the hardest part
- Easy to implement algorithms and protocols
- Harder to handle keys correctly
  - **generate, transfer, store, authenticate, use, update, destroy**
- In the real world, *encryption schemes are defeated not by breaking the algorithm, but by poor key management!!*
- There are two management public-key based algorithms:
  - Key transport - use public/ private keys for encrypting (enveloping) symmetric encryption key
  - Key agreement - exchange public keys for generating a symmetric encryption key





Security topics

"Security is a process, not a product" – B. Schneier

- Computer security
- Network security
- Cloud security
- Security of cyber–physical systems
- Distributed attacks and defense
- Intrusion detection systems
- Cryptography and cryptographic systems
- Cyber-warfare
- Cybersecurity of warfighting systems
- Data trust and assurance
- Cybersecurity education
- Data breaches
- Cybersecurity entrepreneurship
- Security of open systems
- Internet of Things security
- Portable device security
- Sensor network security
- Data privacy
- User privacy
- Cyber forensics
- Malware analysis
- Differential privacy
- Cryptocurrency
- Medical device security
- Voting systems security
- Security testbeds
- Social media cybersecurity topics
- Usability
- Cybersecurity governance, law and policy
- SCADA security
- Insider threats
- Physical security
- **etc. etc. etc…**
- **etc. etc. etc…**

What Can You Do With Cryptography?

"*Cryptography is communication in the presence of adversaries*" – R. Rivest.

- Key exchange
- Encryption
- Digital signatures
- Authentication
- Non-repudiation
- Proof of identity
- Anonymous key distribution
- Blind signatures
- Group signatures
- Secret sharing
- Key escrow
- Public Key Infrastructures (PKI)
- Zero-knowledge proofs
- Simultaneous contract signing
- Digital certified mail
- Electronic Voting
- Digital cash
- **etc. etc. etc.**

Topics in Cryptography

- Data encryption/decryption
  - Symmetric block & stream data encryption
  - Asymmetric data encryption (public key-based encryption)

- Random Number (Bits) Generation
- Message hashing
- Data signing
- Key Agreement (using DH, ECDH, RSA,…)
- Key Wrapping/unwrapping
- Deriving additional cryptographic keys from a secret
- Message Authentication Codes (MACs)
- Elliptic curves cryptography (ECC)

- Quantum cryptography, Identity-based Cryptography, Attribute-based Cryptography, SignCryption, Homomorphic Encryption, Proxy-re-encryption & Proxy re-signing, Group Signatures, etc., etc., etc.

Cryptography Relevant Literature

1. Cryptography classical books
   - A.J. Menezes, P.C. Oorschot, S.A. Vanstone, (1996), ***Handbook of Applied Cryptography***, CRC Press, 816 pagini, ISBN 0849385237.
   - D. R. Stinson, (2005), ***Cryptography: Theory and Practice***, Hall/CRC, 616 pagini, ISBN 1584885084.
   - B. Schneier, (1996), ***APPLIED CRYPTOGRAPHY***, John Wiley & Sons, 784 pagini, ISBN 0471117099.
   - N. Ferguson (2003), B. Schneier, ***Practical Cryptography***, Wiley
2. Cryptography modern books
   - W. Mao, (2003), ***Modern Cryptography: Theory and Practice***, Prentice Hall, 740 pagini, ISBN 0130669431.
   - Jonathan Katz, Yehuda Lindell, (2007), ***Introduction to Modern Cryptography: Principles and Protocols***, Hall/CRC Cryptography and Network Security Series, ISBN-10: 1584885513, ISBN-13: 978-1584885511.
3. Information security books
   - R. J. Anderson, (2001), *Security Engineering: A Guide to Building Dependable Distributed Systems*, Wiley, 640 pagini, ISBN 0471389226.
   - W. Stallings, (2005), ***Cryptography and Network Security*** (4th Edition), Prentice Hall, 592 pagini, ISBN 0131873164.
4. Number theory books
   - V. Shoup, (2004), ***Computational Introduction to Number Theory and Algebra***
5. Research papers
   - Crypto, Eurocrypt, Asiacrypt, International Workshop on Practice and Theory in Public Key Cryptography (PKC), Fast Software Encryption (FSE) şi mai recent Theory of Cryptography Conference (TCC), IACR

Electronic signature ≠ Digital signatures

- Electronic Signatures in **Global and National Commerce Act (E-Sign)** defines:

The term "electronic signature" means an electronic sound, symbol, or process, attached to or logically associated with a contract or other record and executed or adopted by a person with the intent to sign the record.

- Electronic signature is a proprietary format (there is no standard for electronic signatures) that is an electronic data, such as a digitized image of a handwritten signature, a symbol, voiceprint, etc., that identifies the author(s) of an electronic message. An electronic signature is vulnerable to copying and tampering, making forgery easy. In many cases, requires proprietary software to validate the e-signature.

- Digital signature, often referred to as ***advanced or standard electronic signature, is a sub group within electronic signatures*** which provide the highest form of signature and content integrity as well as universal acceptance.
  - Is based on Public Key Infrastructure (PKI) and
  - Is a result of a cryptographic operation that
  - Guarantees **signer authenticity**, **data integrity** and **non-repudiation** of signed documents.

- Digital signature cannot be copied, tampered or altered.
- Digital signatures made within one application (e.g. DocuSign, Adobe® PDF) can be validated by others using the same applications

Security Mechanisms

Basic building blocks are used:

- ***Encryption***
  - used to provide confidentiality, **can provide authentication and integrity protection**
- **Hash/ checksums algorithms**
  - **used to provide integrity protection, can provide authentication (MACs, HMAC)**
- ***Digital signatures***
  - **used to provide authentication, integrity protection, and non-repudiation**

One or more security mechanisms are combined to provide a security service

Digital Signatures-based Mechanisms

- **Security objectives:**
  - Data Integrity
  - Origin Authentication
  - Data Authentication
  - Origin Non-repudiation
  - Data Non-repudiation

- **Digital signatures provide the ability to:**
  - verify author, date & time of signature
  - authenticate message contents
  - be verified by third parties to resolve disputes

Digital Signatures-based Mechanisms

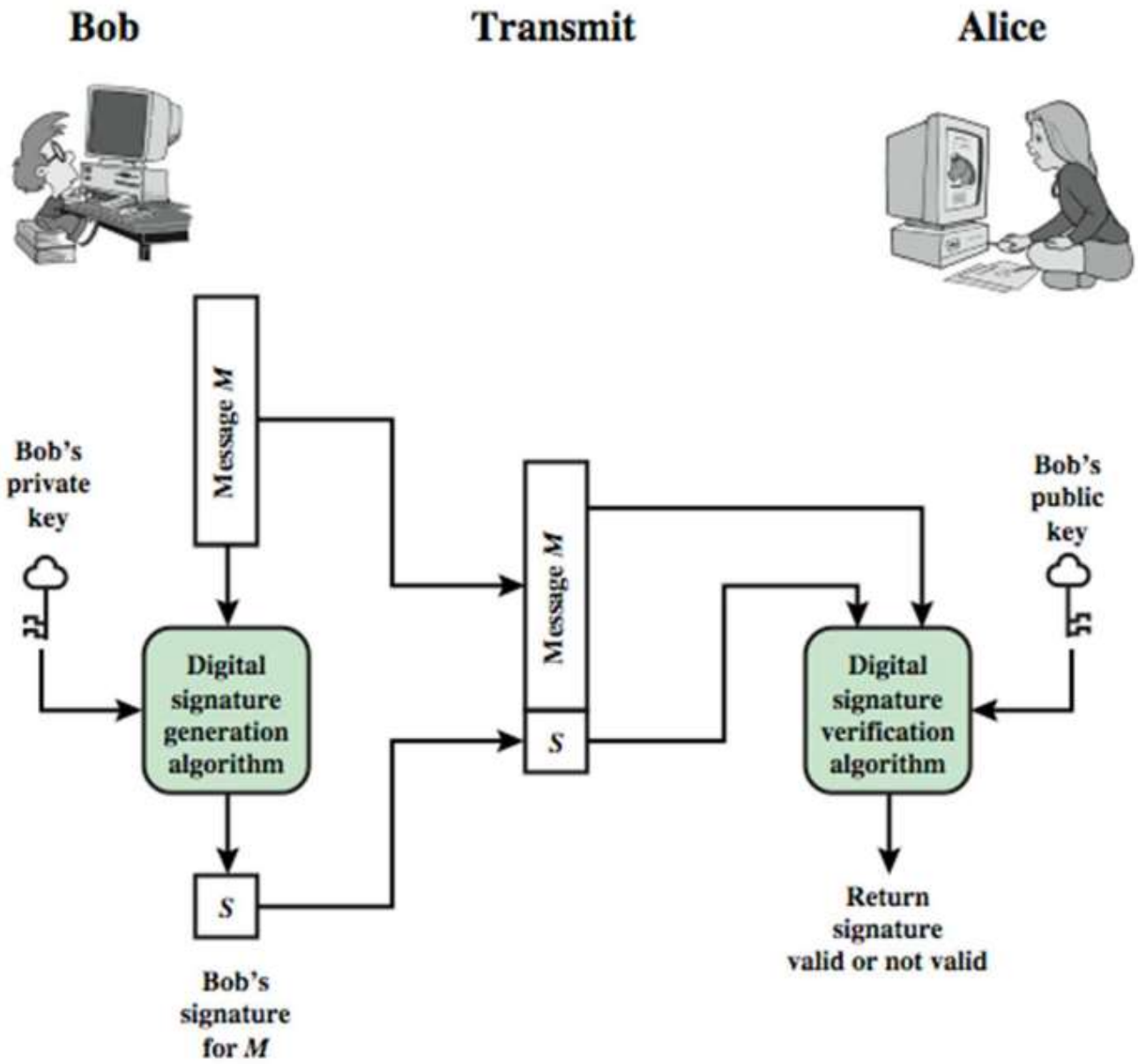
- **Algorithms: RSA, DSA, ECDSA (+SHA2, SHA3, etc.)**
  - **Combines a hash with a digital signature algorithm**

- **Recommended *Key Lengths* for electronic signatures:**
  - **2048+bits key RSA**
  - **2048+ bits key DSA**
  - 224, **256+ bits Elliptic Curves-based DSA (ECDSA)**

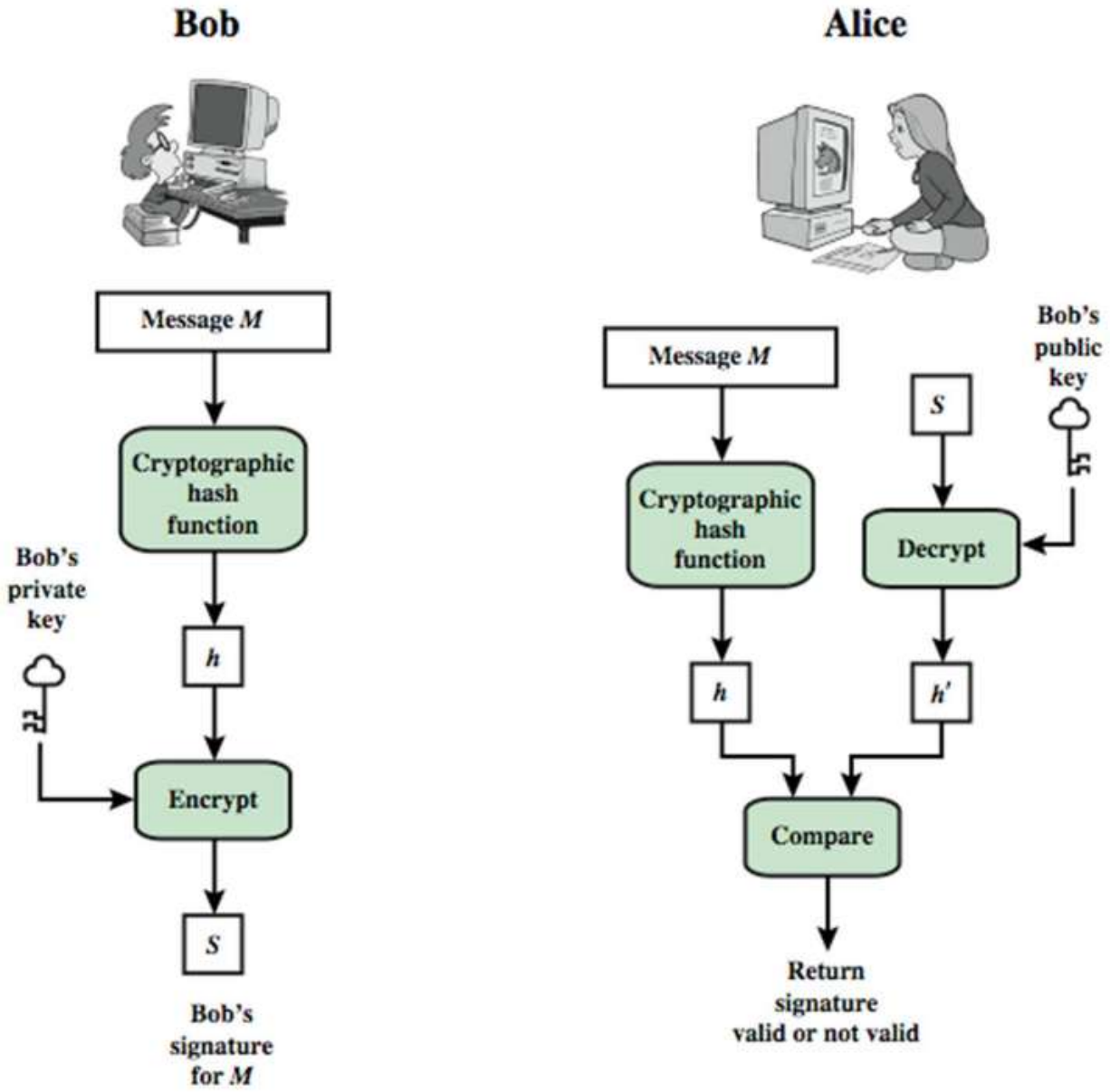
Digital Signatures Model

- **Signatures creation (generation)**
  - **Using signer-user's private key**
- **Signature verification (validation)**
  - **Using signer-user's public key**

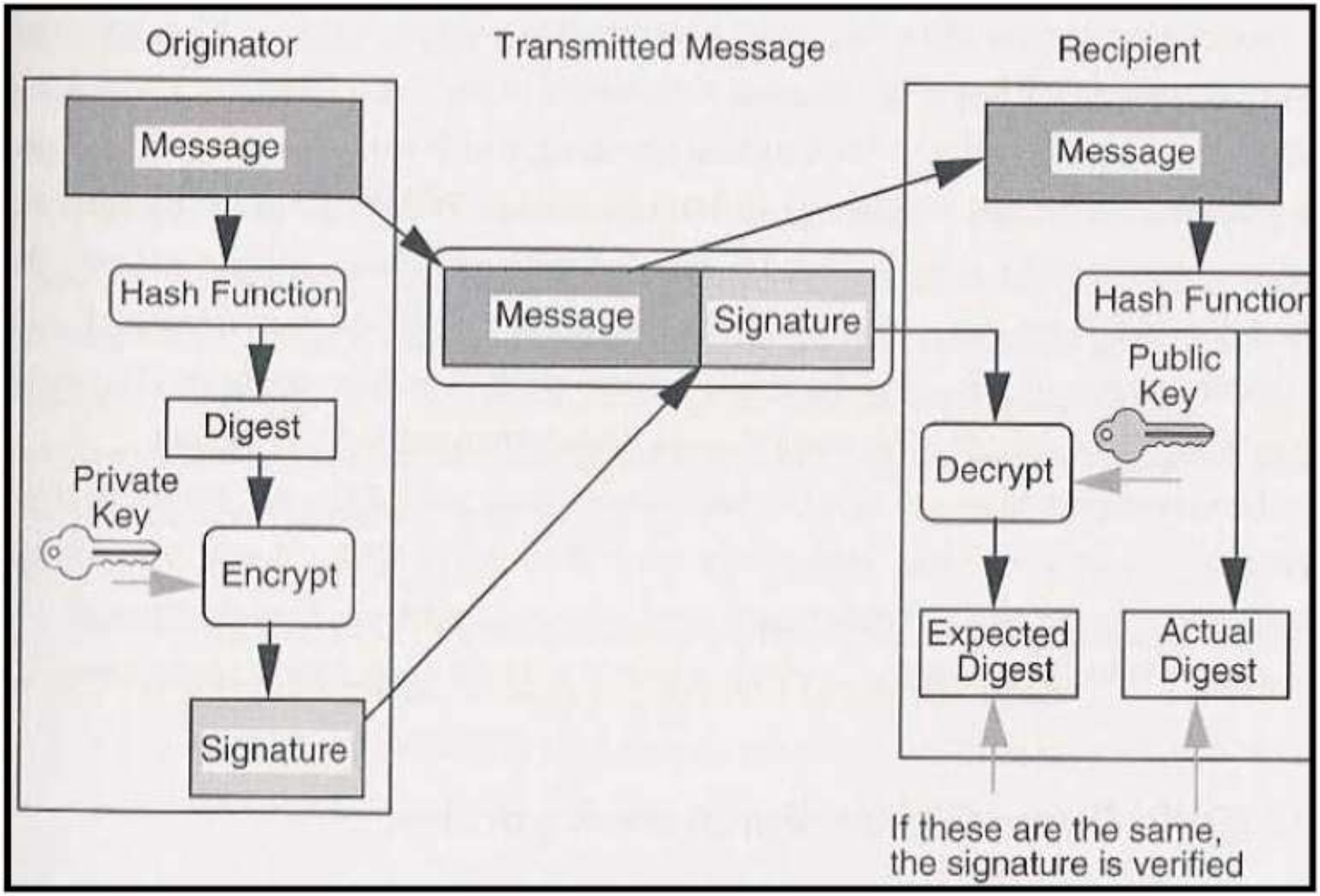
Digital Signatures Model (cont.)



Digital Signatures Model (cont.)



Digital Signatures creating & verifying



Digital Signatures Requirements

- Must depend on the message to be signed
- Must use information unique to signer (signer's private key)
  - to prevent both forgery and denial
- Must be relatively easy to produce
- Must be relatively easy to recognize & verify
- Must be computationally infeasible to forge
  - with new message for existing digital signature
  - with fraudulent digital signature for given message
- Have to be practical to save digital signature in storage

Digital Signatures

- Involve only signer & verifier
- Assumed receiver has signer's public-key
- Digital signature made by sender signing entire message (or a hash of it) with sender's private-key
- The signature can be encrypted using receiver's public-key (optional)
  - It's important to sign first then encrypt message & signature
- Security depends on signer's private-key

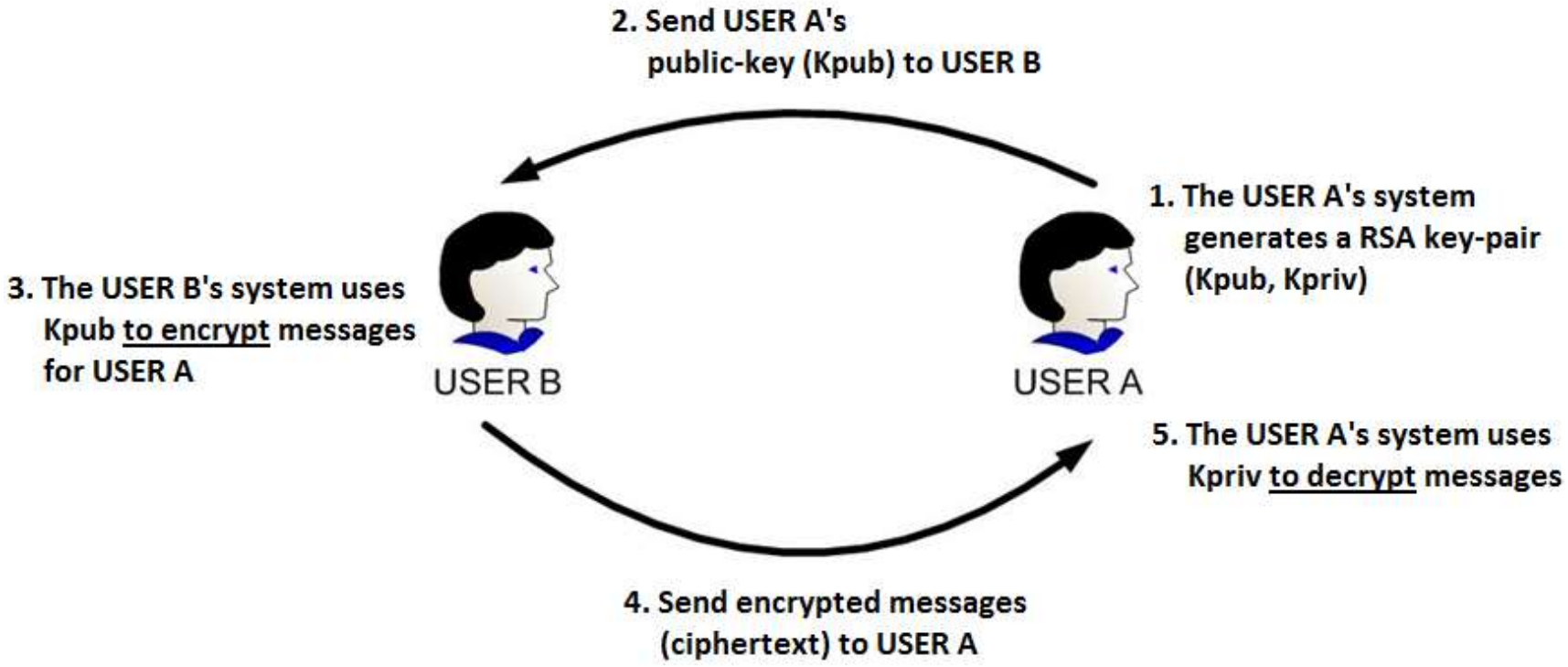
RSA

- Proposed by **R**ivest, **S**hamir & **A**dleman (MIT) in 1977
- The best known & widely used **public-key** scheme (encryption & signature)
- It's based on exponentiation in a finite (Galois) field over integers modulo a prime
  - nb. exponentiation takes $O((\log n)^3)$ operations (**easy**)
- RSA uses large integers (e.g. 2048 bits, 3192 bits, 4096 bits, ….)
- The security exists due to cost of factoring large numbers
  - nb. Integers factorization takes $O(e^{\log n \log \log n})$ operations (**hard**)

RSA cryptosystem

Encryption based on RSA algorithm

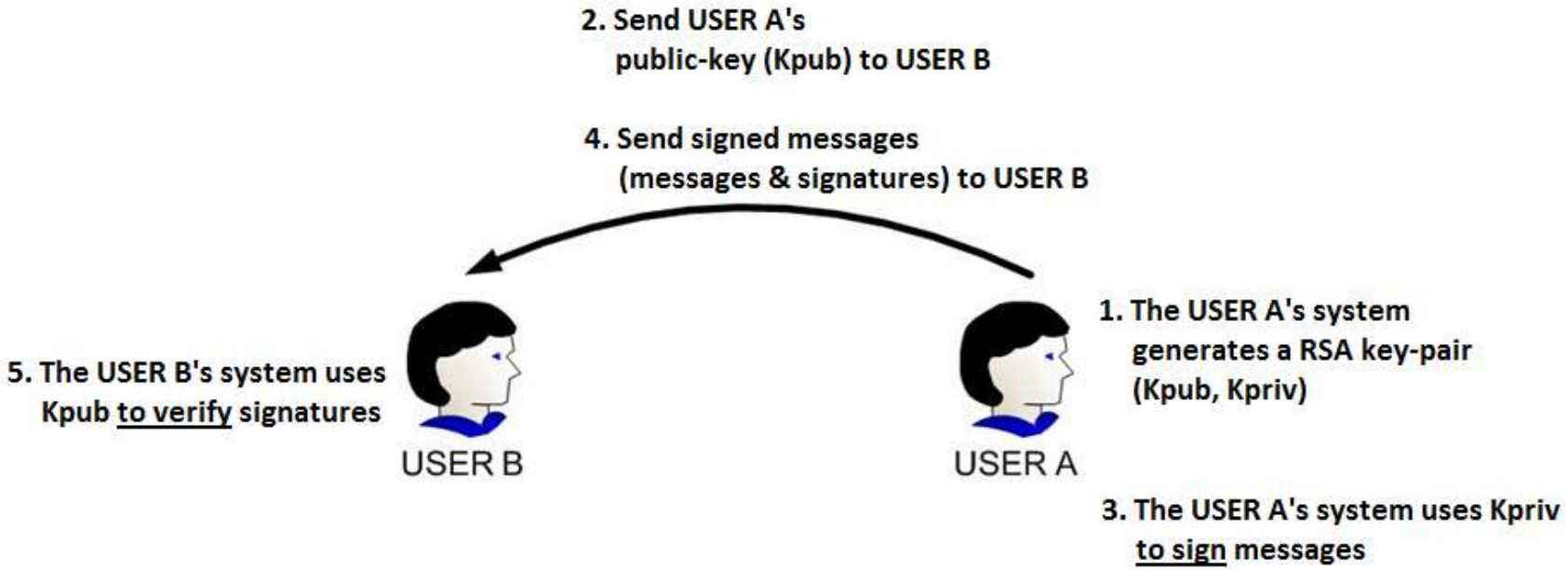
- Generation of the RSA key-pair
- Usage of the RSA key-pair to **encrypt/decrypt** messages



RSA cryptosystem

Signing with RSA algorithm

- Generation of the RSA key-pair
- Usage of the RSA key-pair to **sign/verify** messages



Digital Signatures using RSA

- RSA key-pair generation algorithm
- RSA signatures generation algorithm
- RSA signature verification algorithm

Digital Signatures using RSA (cont.)

RSA key-pair generation algorithm:

1. Generate two different prime numbers p, q
2. Computes $n = p \cdot q$ and $\Phi(n) = (p-1) \cdot (q-1)$
3. Choses an integer number e with $1 < e < \Phi$ and $gcd\,(e, \Phi) = 1$
4. Computes d so that $1 < d < \Phi$ and $d \cdot e \equiv 1\,(mod\ \Phi)$ – using Euclid's extended algorithm
5. The public-key of the USER A is the pair of integers (n, e)
6. The private-key of the USER A is the pair of integers (n, d)

Digital Signatures using RSA (cont.)

RSA key-pair generation algorithm:

1. Generate two different prime numbers p, q
2. Compute $n = p \cdot q$ and $\Phi(n) = (p-1) \cdot (q-1)$
3. Chose an integer number e with $1 < e < \Phi$ and $gcd\,(e, \Phi) = 1$
4. Compute d so that $1 < d < \Phi$ and $d \cdot e \equiv 1\,(mod\ \Phi)$ – using Euclid's extended algorithm
5. The public-key of the USER A is the pair of integers (n, e)
6. The private-key of the USER A is the pair of integers (n, d)

Note (terminology):

- p, q : the prime numbers (big numbers)
- n : the modulus of the key-pair
- e : the public exponent
- d : the private exponent
- $Kpub = (n, e)$ – public key, used to encrypt messages/ to verify signatures
- $Kpriv = (n, d)$ – private key; used to decrypt cipher texts/ to sign messages

Digital Signatures using RSA (cont.)

RSA signatures <u>generation</u> algorithm:

To sign a message $m \in M$, *the USER A* will make the next computations:

1. **Compute $\tilde{m} = h(m)$ using a collision resistant hash function h**
2. **Compute $s = \tilde{m}^d \pmod n$**

Note:

- **The signature of USER A on the message m is the integer s**
- **To verify the signature s on the message m, one must have $(m, s, (n, e))$**

Digital Signatures using RSA (cont.)

RSA signatures verification algorithm:

To verify a signature s on the message $m \in M$, any USER *B* will make the next computations:

1. **Get the USER A's public-key (n, e)**
2. **Compute $\widetilde{m} = s^e \pmod n$. This will extract the initial hash from signature**
3. **Compute $\widetilde{m'} = h(m)$ using the same collision resistant hash function h**
4. **Compare $\widetilde{m} = \widetilde{m'}$**
   a) **If $\widetilde{m} = \widetilde{m'}$, then the signature s on the message m is valid.**
   b) **Otherwise the signature is invalid.**

Note:

- **To verify the signature s on the message m, someone must have $(m, s, (n, e))$**
- **Signature generation algorithm returns the integer s**
- **Signature verification algorithm returns *true/false***
- **If *false*, something is wrong in the tuple (m, s, n, e)**

Digital Signatures using RSA (cont.)

- **Security: The integer number factorization problem (IF) – *hard* problem :**
  - Let n a composite integer number, $n = p \cdot q$, where p, q are prime big-numbers (length of 1024 bits for each)
  - Knowing n, it's hard to find its divisors p, q

- **Considerations:**
  - You know n but you don't know $\Phi(n)$
  - If you know $\Phi(n)$, you can find easily the private key $d = e^{-1} \, (mod \, \Phi(n))$
  - Remember that $\Phi(n) = (p-1) \cdot (q-1)$
  - You know n but you can find $\Phi(n)$ only and only if you find p, q the divisors of n

Public-Key Cryptography Standards (PKCS)

- **PKCS#1 – RSA algorithm based cryptography standard**
- PKCS#2 – Cryptographic digest encryption (included into #1)
- PKCS#3 – Diffie-Hellman Key Agreement standard
- PKCS#4 – RSA key format (included into #1)
- PKCS#5 – Password based encryption (PBE) standard
- PKCS#6 – X.509 certificate extended syntax – extensions – standard
- PKCS#7 – Cryptographic Message Syntax (CMS) standard
- PKCS#8 – Private keys format standard
- PKCS#9 – Attributes types standard
- PKCS#10 – Certificate request format standard
- PKCS#11 – API interface standard for handling hardware cryptographic devices
- PKCS#12 – Software tokens syntax standard
- PKCS#13 – ECC based cryptography– in progress
- PKCS#14 – TRNG and PRNG standard – in progress
- PKCS#15 – Information syntax for cryptographic token standard

Digital Signature Standard – DSS (DSA)

- US Govt approved signature scheme
- It's a public-key cryptography based technique
- Designed by NIST & NSA in early 90's
- Published as FIPS-186 in 1991
- Revised in 1993, 1996 & then 2000
- Uses the SHA hash algorithm
- DSS is the standard, DSA is the algorithm
- FIPS 186-2 (2000) includes alternative RSA & elliptic curve signature variants
- DSA is digital signature only (unlike RSA)

Digital signatures using DSA

- DSA = Digital Signature Algorithm
- Proposed by NIST in 1991 as DSS = Digital Signature Standard (FIPS-186)
- DSA allows signature only – it DOES NOT allow data encryption

- Security: The discrete logarithm problem (DLP) – *hard* problem :
  - Let p a prime big integer number, g a integer number, $1 < g < p - 1$. Let $y = g^x \pmod p$
  - Find x if you know p, g, y

- DSA scheme involves also 3 algorithms:
  - DSA key-pair generation algorithm
  - DSA signature generation algorithm
  - DSA signature verification algorithm

Digital Signatures using DSA (cont.)

DSA key-pair generation algorithm:

Assume

- Key length = 2048 bits
- The key will be used with a hash function on 256 bits (e.g. SHA256)

1. Generate a random big number number p, such that $2^{L-1} < p < 2^{L}$, where $1024 < L < 2048$
2. Choose a prime integer q , divisor of $(p-1)$, such that $2^{255} < q < 2^{256}$
3. Compute $g = \alpha^{p-1} \ (mod\ p)$ where α is an integer $1 < \alpha < p-1$, such that $\alpha^{(p-1)/q} \ (mod\ p) > 1$
4. Generate randomly an integer number x such that $1 \leq x \leq q-1$
5. Compute $y = g^{x} \ (mod\ p)$

6. The private-key is the integer x
7. The public-key is the integer y
8. Parameters (p, q, g) are *domain parameters*
   - These may be used in common by multiple users

Digital Signatures using DSA (cont.)

DSA signatures generation algorithm:

To sign a message $m \in M$, *the USER A* will make the next computations:

1. **Compute $h(m)$ using an appropriate collision resistant hash function *h***
2. **Select a random integer number k such that $1 \leq k \leq q-1$**
3. **Compute $r = (g^k \ (mod\ p))(mod\ q)$. If $r = 0$ jump again to step 2**
4. **Compute $s = \left(k^{-1} \cdot (h(m) + x \cdot r)\right)(mod\ q)$. If $s = 0$ jump again to step 2**

5. **The signature on message m is the pair (r, s)**

Digital Signatures using DSA (cont.)

DSA signatures generation algorithm:

To sign a message $m \in M$, *the USER A* will make the next computations:

1. **Compute $h(m)$ using an appropriate collision resistant hash function *h***
2. **Select a random integer number k such that $1 \leq k \leq q-1$**
3. **Compute $r = (g^k \ (mod\ p))(mod\ q)$. If $r = 0$ jump again to step 2**
4. **Compute $s = \left(k^{-1} \cdot (h(m) + x \cdot r)\right)(mod\ q)$. If $s = 0$ jump again to step 2**

5. **The signature on message m is the pair (r, s)**

Note:

- The size of DSA signatures does not depend on the key size or message size.
- The size of DSA signatures depends on the q size which is same with the hash function size
- (E.g.) If we sign a message using SHA256 hash function, the signature size will be of 512 bits
- The DSA signature algorithm includes a random factor (k value). This means that two instances of the DSA to sign the same message with the same key will lead to different output.
- Discussion regarding the security of k

Digital Signatures using DSA (cont.)

DSA signatures verification algorithm:

To verify a signature (r, s) on the message $m \in M$, any USER *B* will make the next computations:

1. **Achieve the signer public-key y and the corresponding domain parameters (p, q, g). The public-key must be authentic.**
2. **Check if the r, s values are within the range $[1, q-1]$**
3. **Compute $h\,(m)$ using same hash function *h* used by the signer**
4. **Compute $w = s^{-1}\,(mod\ q)$**
5. **Compute $u_1 = (h(m) \cdot w)(mod\ q)$ and $u_2 = (r \cdot w)(mod\ q)$**
6. **Compute $v = \big((g^{u1} \cdot y^{u2})(mod\ p)\big)\,(mod\ q)$**
7. **Compare $v = r$**
   a) **If $v = r$, then the signature (r, s) on the message m is valid.**
   b) **Otherwise the signature is invalid.**

Digital Signatures using DSA (cont.)

DSA signatures verification algorithm:

The proof:

$$v = \left((g^{u1} \cdot y^{u2})(mod\ p)\right)(mod\ q) = \left(\left(g^{h(m)\cdot w} \cdot y^{r\cdot w}\right)\right)(mod\ p)(mod\ q) =$$

$$\left(\left(g^{h(m)\cdot s^{-1}} \cdot g^{x r\cdot s^{-1}}\right)\right)(mod\ p)(mod\ q) = g^{h(m)\cdot s^{-1}+r\cdot x\cdot s^{-1}}(mod\ p)(mod\ q) =$$

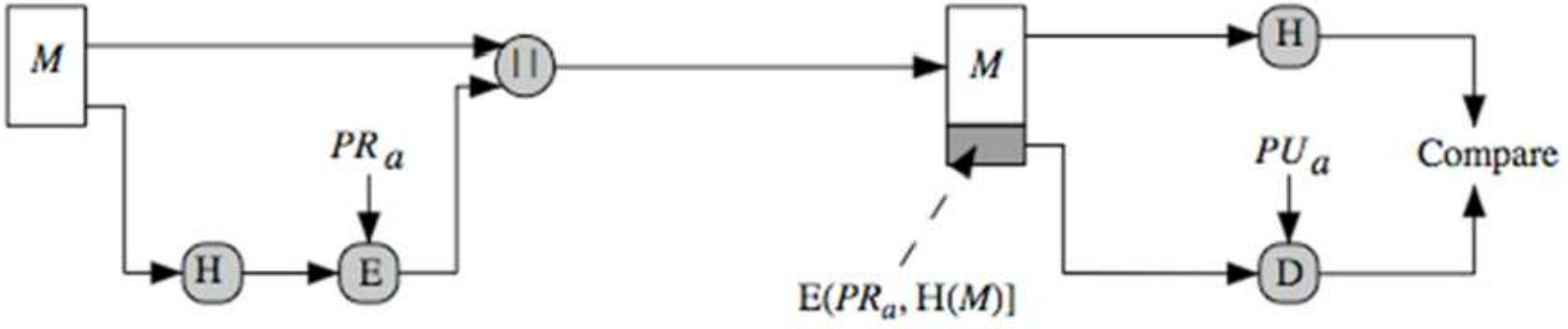
$$g^{s^{-1}\cdot(h(m)+r\cdot x))}(mod\ p)(mod\ q) = g^{s^{-1}\cdot s\cdot k}(mod\ p)(mod\ q) =$$

$$g^{k}(mod\ p)(mod\ q) = r$$

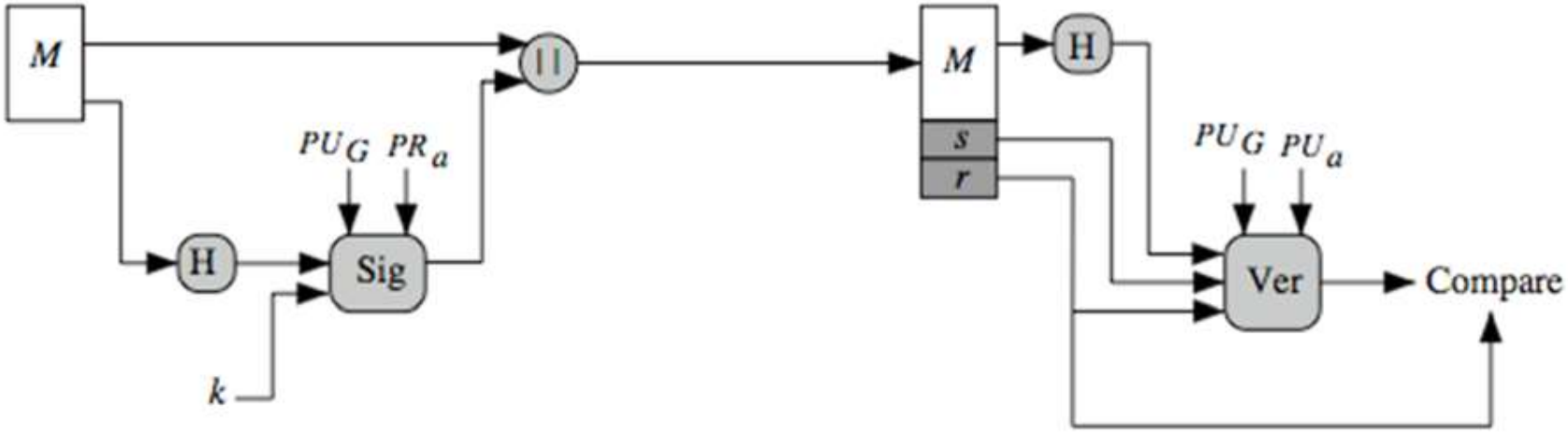
Notes:

1. **Signing operation uses the private key x**
2. **Verification operation uses public key y**
3. **You can NOT find x knowing $(y,(g,p,q),(r,s))$ (DLP)**
4. **You can NOT find k knowing $(y,(g,p,q),(r,s))$ (DLP)**

Schematics of RSA and DSA



(a) RSA Approach



(b) DSS Approach

Signature Algorithm types

- **Two digital algorithm types:**
  - Digital signature with message recovery (ex. RSA can be if not hashing)
  - Digital signature with appendix (ex. DSA, RSA-with-hashing)

- **Usually, sign a message digest (hash)**
  - RSA is converted also to a digital signature with appendix

Public-Key Cryptography Standards (PKCS)

- **PKCS#1 – RSA algorithm based cryptography standard**
- PKCS#2 – Cryptographic digest encryption (included into #1)
- PKCS#3 – Diffie-Hellman Key Agreement standard
- PKCS#4 – RSA key format (included into #1)
- PKCS#5 – Password based encryption (PBE) standard
- **PKCS#6 – X.509 certificate extended syntax – extensions – standard**
- **PKCS#7 – Cryptographic Message Syntax (CMS) standard**
- **PKCS#8 – Private keys format standard**
- **PKCS#9 – Attributes types standard**
- **PKCS#10 – Certificate request format standard**
- **PKCS#11 – API interface standard for handling hardware cryptographic devices**
- **PKCS#12 – Software tokens syntax standard**
- PKCS#13 – ECC based cryptography– in progress
- PKCS#14 – TRNG and PRNG standard – in progress
- PKCS#15 – Information syntax for cryptographic token standard

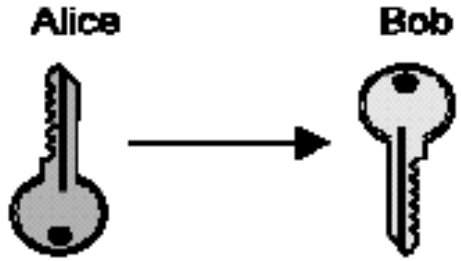
Digital Signatures-based Authentication

- Digital Signatures
- Public Key Distribution (key-management aspects)
- Certification Authorities (CAs)
- Digital certificates (X.509)
- PKI – Public Key Infrastructures
  - PKI components: CAs, RAs, Repositories, Users
- Certificates validation process
  - Certificates path validation (be sure about the certificates authenticity)
  - Certificates status checking (revoked or not)
- Certificate Revocation Lists (CRLs)
- Builds (discover) and validates the certificate path
- Other components
  - Time-stamping Authority, OCSP Authority , Signing Authority

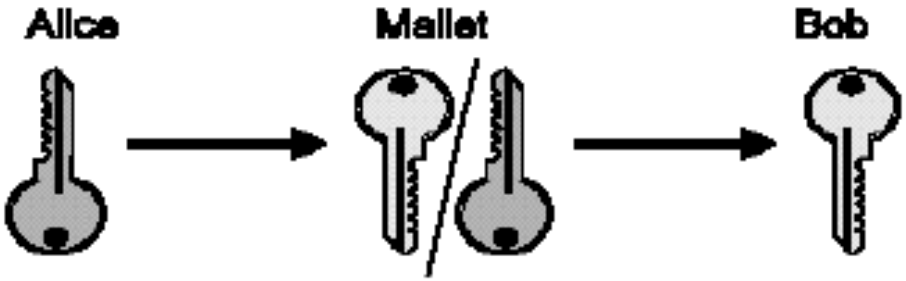
Public Key Distribution

- **The big problem: Man in the Middle !!!**

Alice retains the private key and sends the public key to Bob

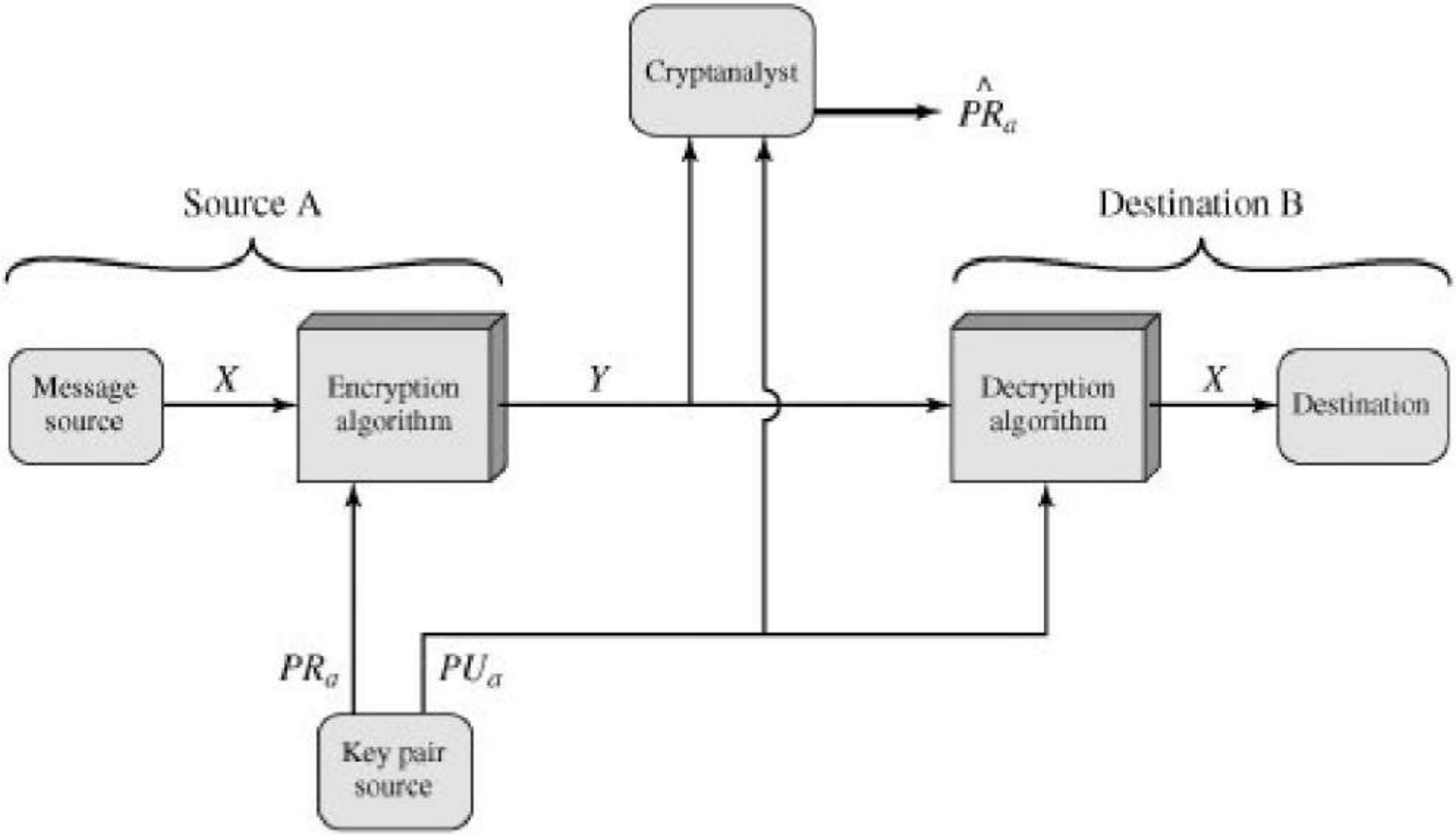


Mallet intercepts the key and substitutes his own key



Mallet can decrypt all traffic and generate fake signed message

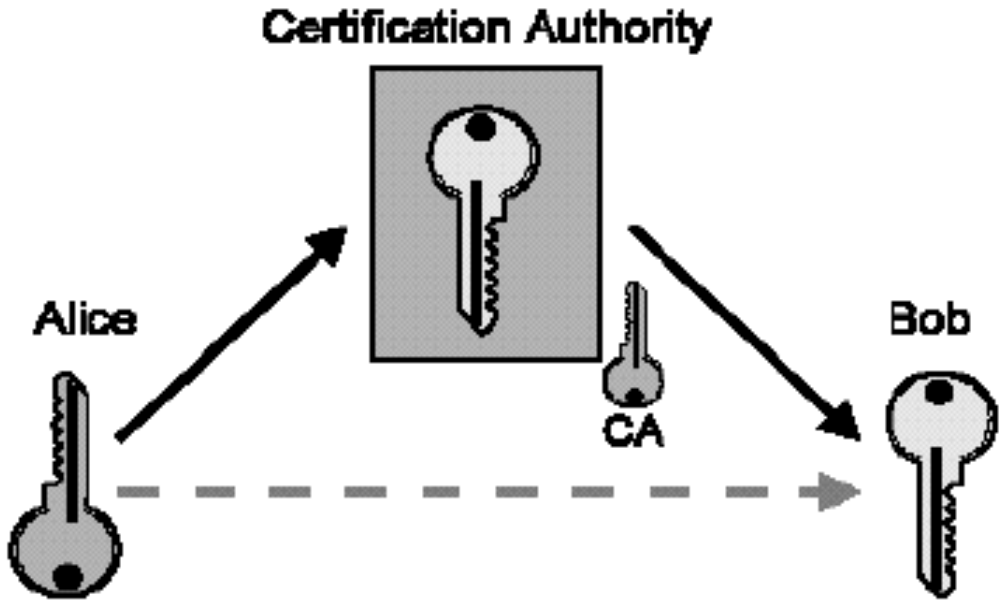
Generating fake signatures



Public Key Distribution (contd.)

- **A trusted third party should authenticates the public key**

A certification authority (CA) solves this problem



CA signs Alice's key to guarantee its authenticity to Bob

- Mallet can't substitute his key since the CA won't sign it

Digital Certificate (contd.)

- Is a person really who claim?

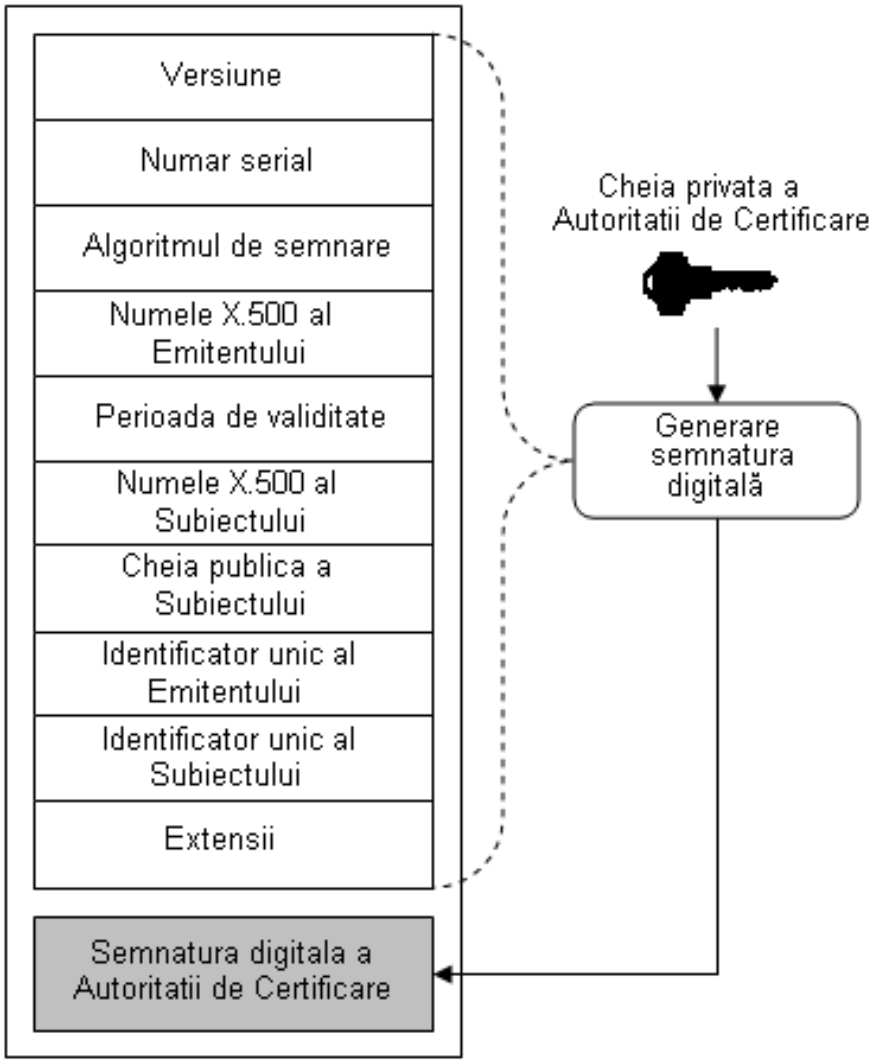
- How do you know that the public key you got from a person (entity) really belongs to this person

- Solution: CERTIFICATE – like an *Information Highway Driver License*

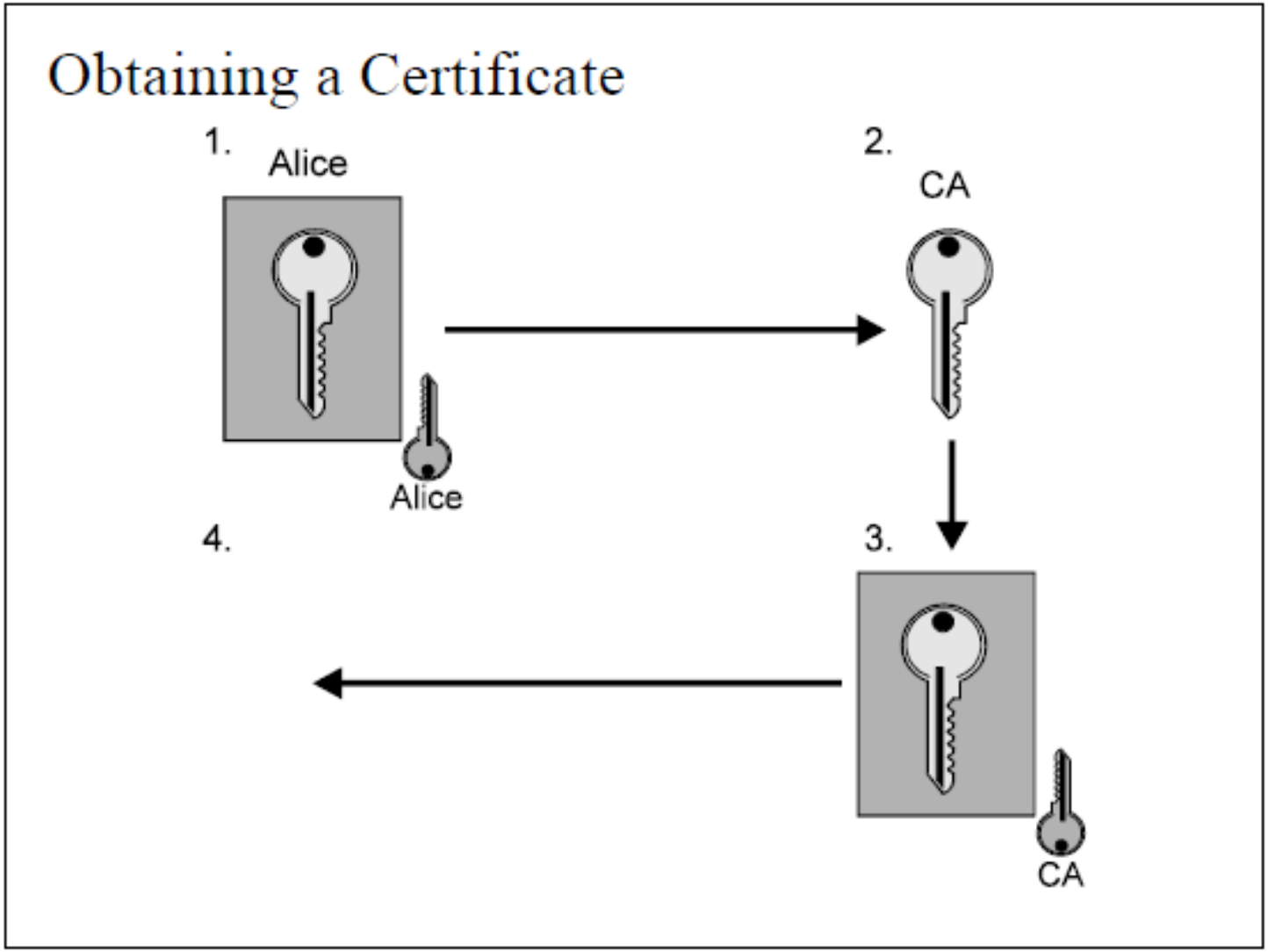
Digital Certificate X.509 v3

Certificate contents:

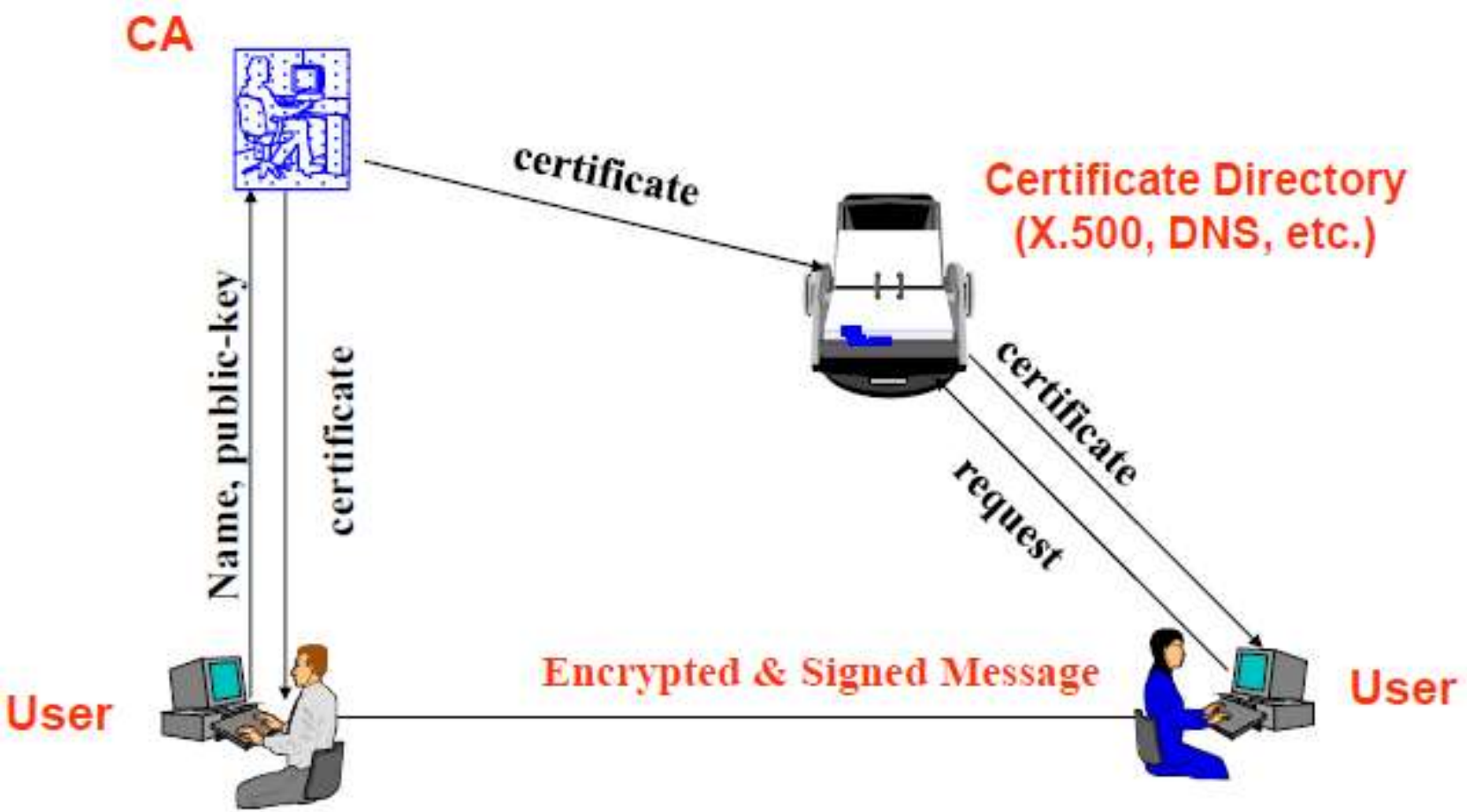
- Version number is 3
- Serial number is a monotonically increasing integer value (guaranties the unicity of serial number for issuing CA)
- Issuer name is populated with X.500 distinguished name
- Subject public key corresponds to a standard algorithm
- Signature field identifies a standard signature algorithm.



Digital Certificate X.509 v3

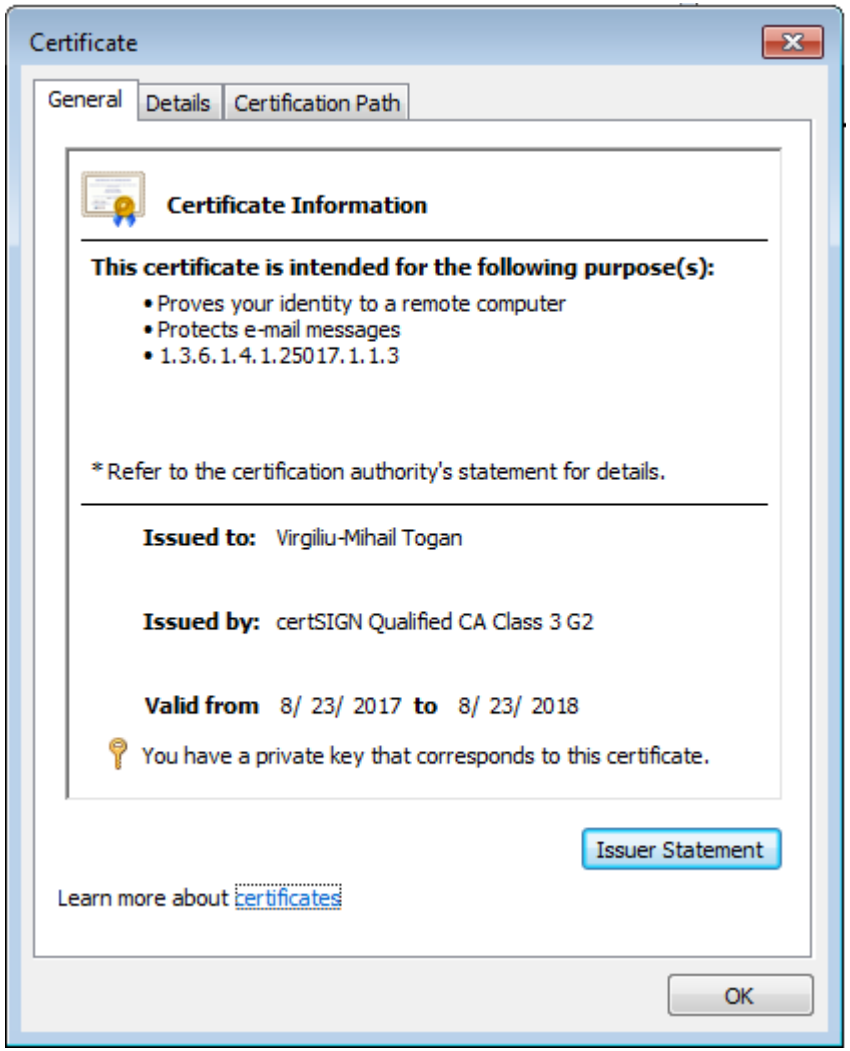


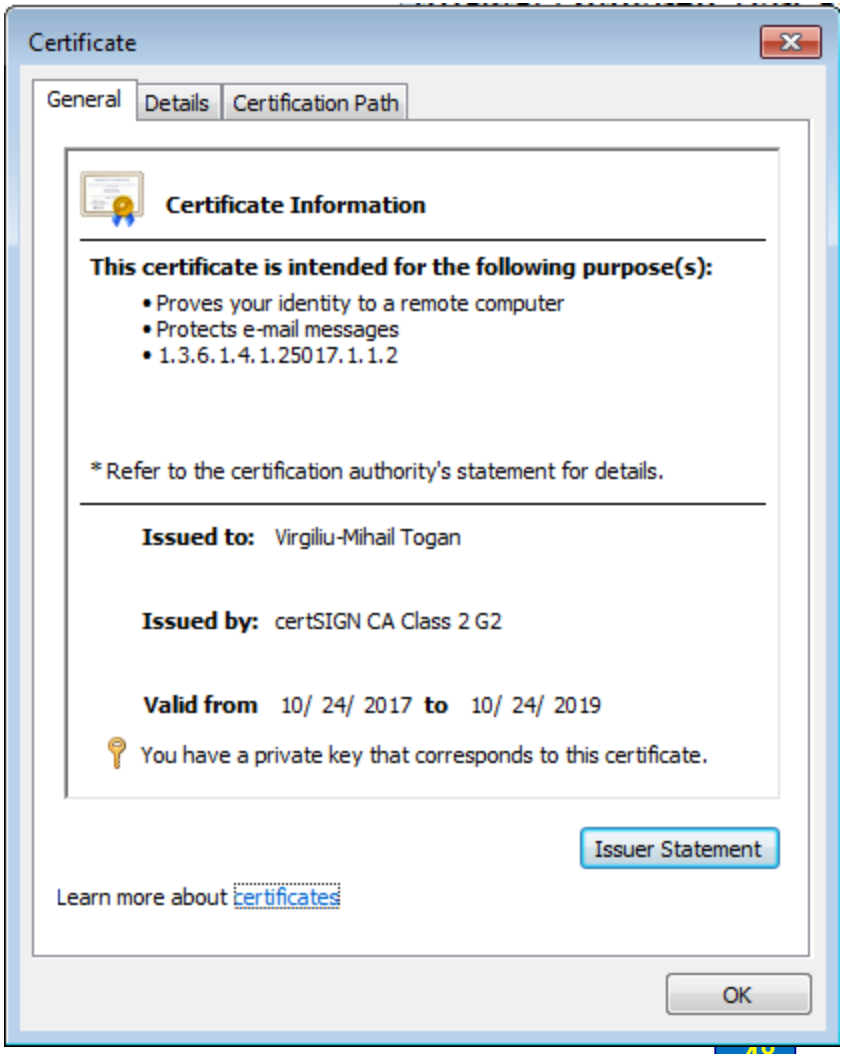
Workflows in PKI



Examples of Digital Certificates X.509 v3

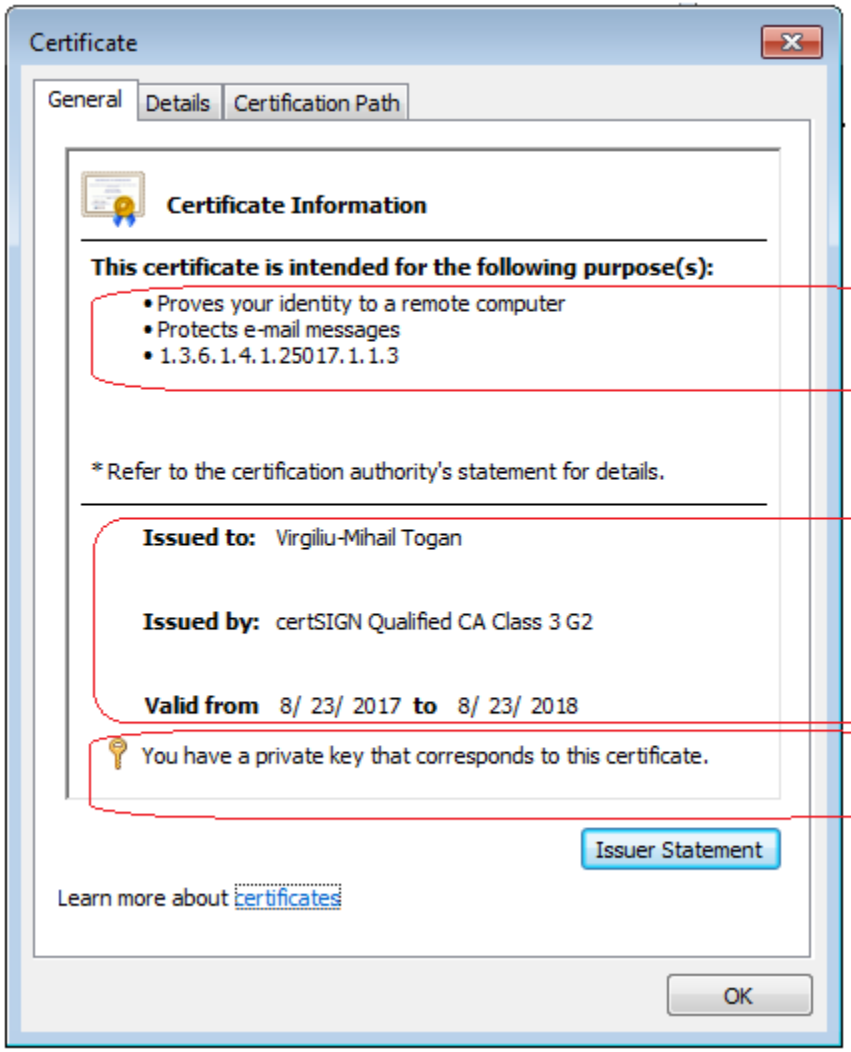
(using Windows built-in Certificate Viewer)





Example of Digital Certificates X.509 v3 (contd.)

(using Windows built-in Certificate Viewer)



There are several categories of information:

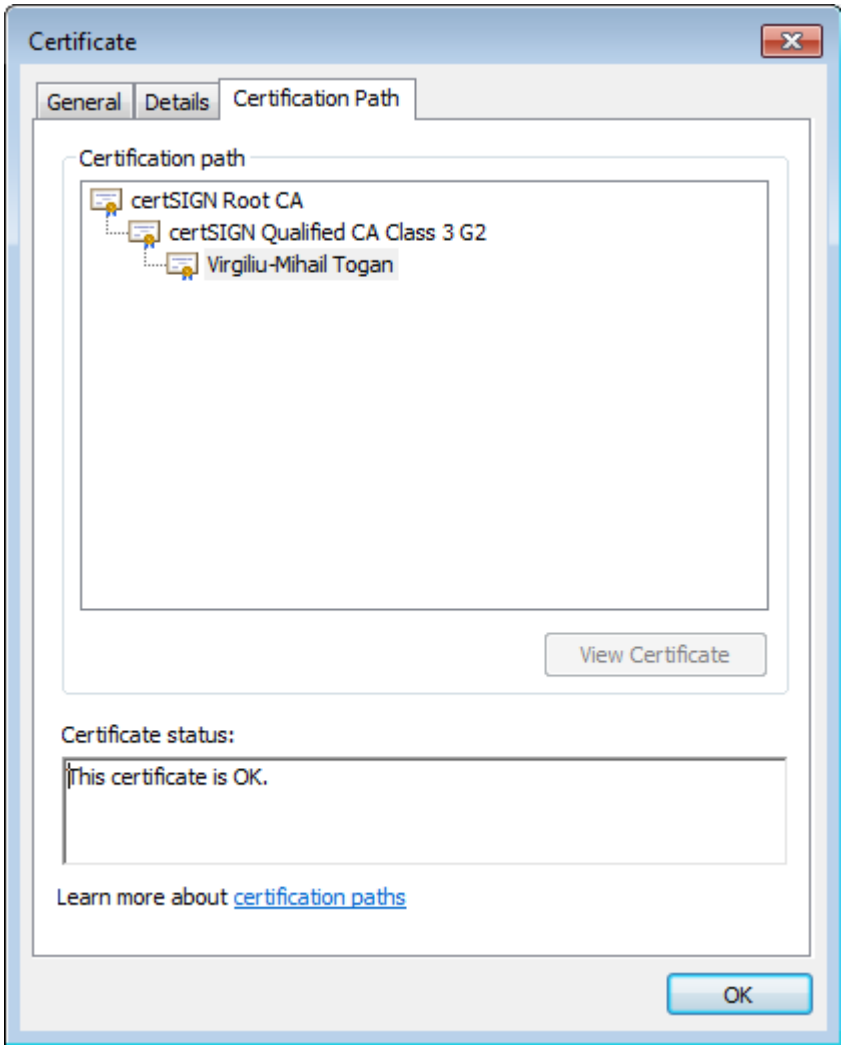
The main scope of certificate

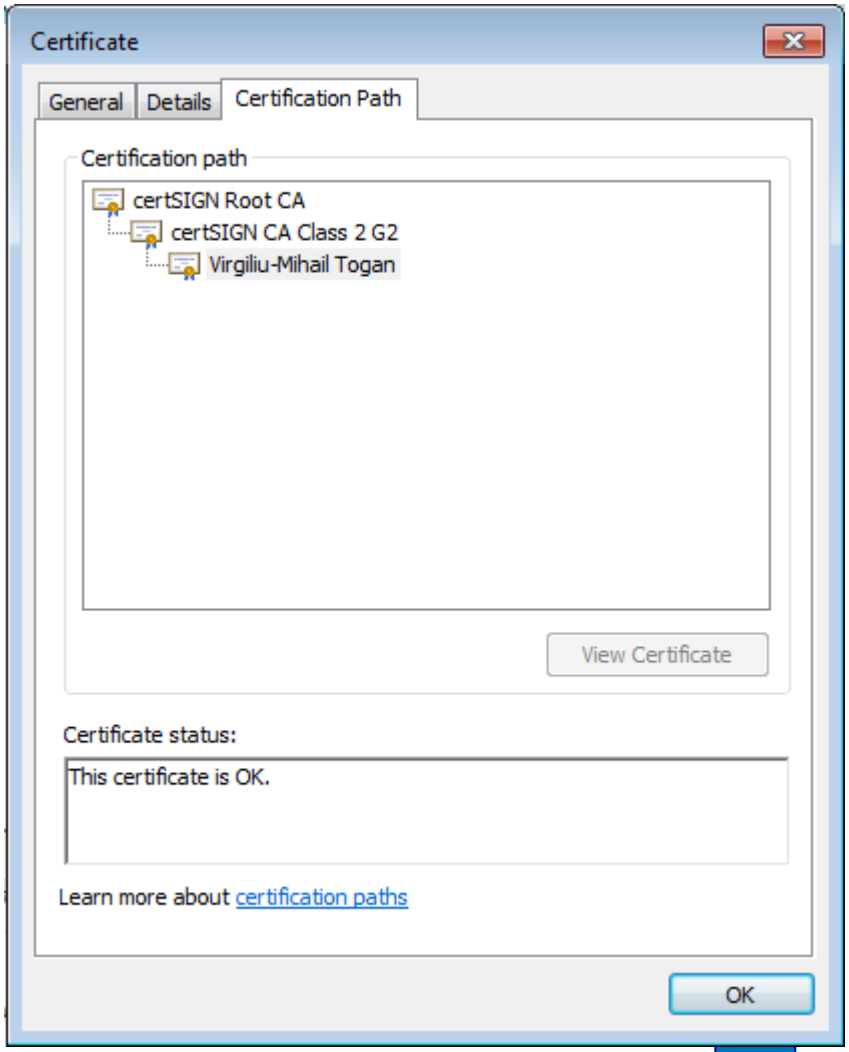
The SUBJECT of certificate
The ISSUER of certificate
The validity of certificate

If there is a registered private-key, pairing with this public key

Certificate Chains

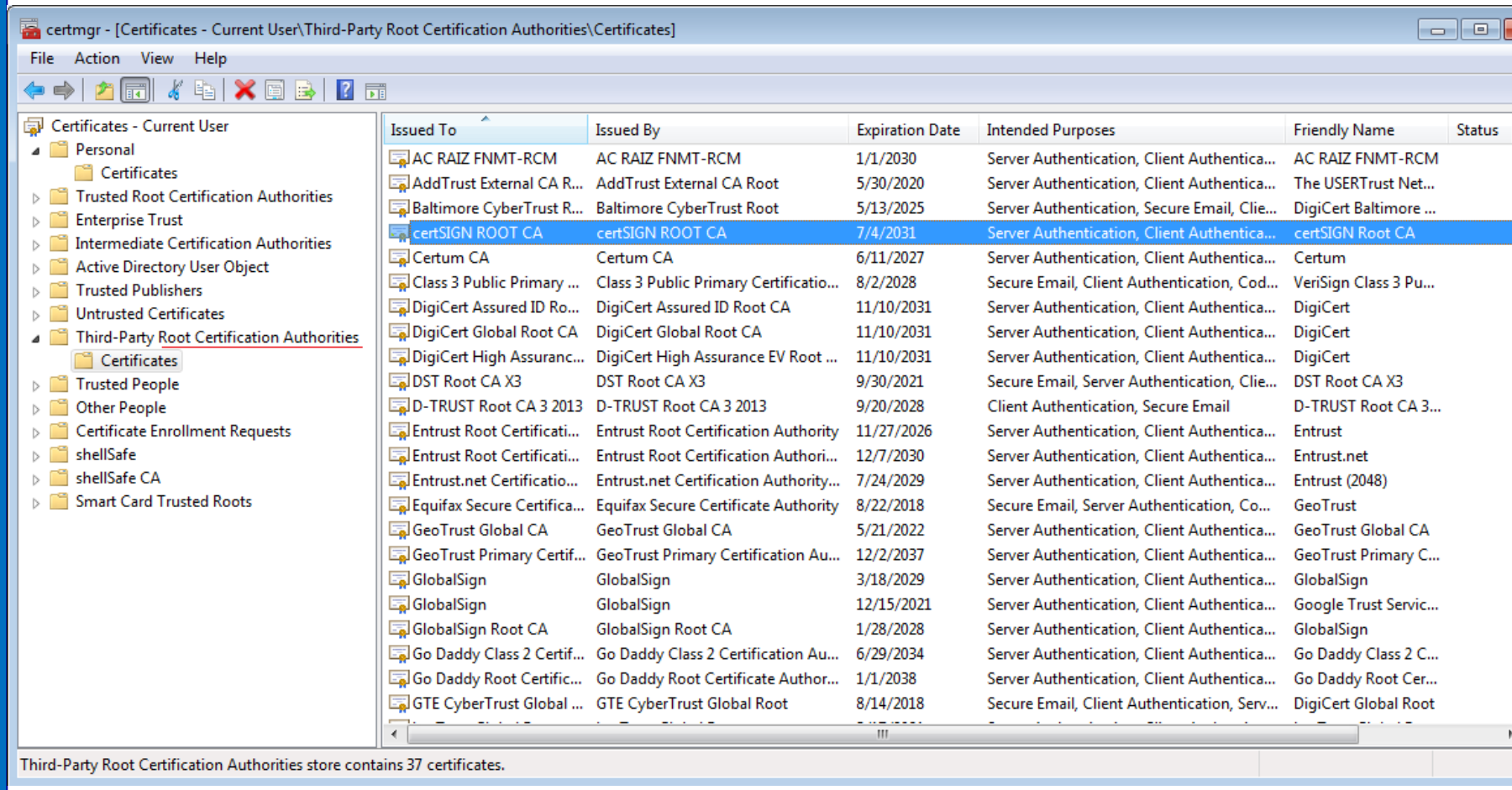
- Each certificate has a *certification-path starting from a ROOT-CA*
- Usually, Root-CA certificate is a trusted point in the *system*





Digital Certificate X.509 v3 (contd.)

- **Trusted certificates (trust anchors) in Windows**

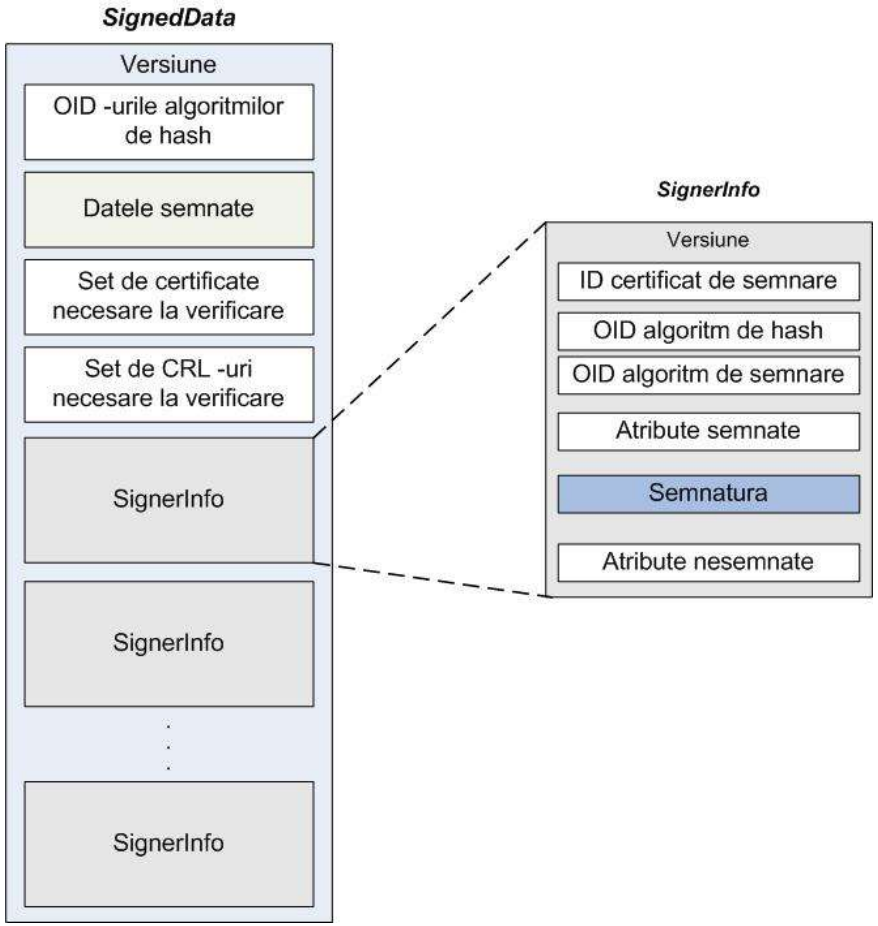


Using signatures, certificates & PKI in the real world

- Document signing (ETSI, PKCS#7 formats)
  - Signatures with legal value
- Email encryption & signing (S/MIME)
- Transport-layer VPNs (using SSL)
- Network-layer VPNs (using IPSec)
- Web servers authentication (HTTPS)
- Web servers & Client mutual authentication (HTTPS)
- TCP servers & Clients authentication (TLS)
- User authentication to services (databases, etc.)
- Network/IT Infrastructure services access (smartcard logon)
- …

Document signing in the real world

- **PKCS#7 signature profile**




```
SignedData ::= SEQUENCE
{
    version             CMSVersion,
    digestAlgorithms    DigestAlgorithmIdentifiers,
    encapContentInfo    EncapsulatedContentInfo,
    certificates        [0] IMPLICIT CertificateSet OPTIONAL,
    crls                [1] IMPLICIT RevocationInfoChoices OPTIONAL,
    signerInfos         SignerInfos
}
```

```
SignerInfos ::= SET OF SignerInfo
```

```
SignerInfo ::= SEQUENCE
{
    version Version,
    issuerAndSerialNumber       IssuerAndSerialNumber,
    digestAlgorithm             DigestAlgorithmIdentifier,
    authenticatedAttributes     [0] IMPLICIT Attributes OPTIONAL,
    digestEncryptionAlgorithm   DigestEncryptionAlgorithmIdentifier,
    encryptedDigest             EncryptedDigest,
    unauthenticatedAttributes   [1] IMPLICIT Attributes OPTIONAL
}

EncryptedDigest ::= OCTET STRING
```

Document signing in the real world

Relevant legislative framework

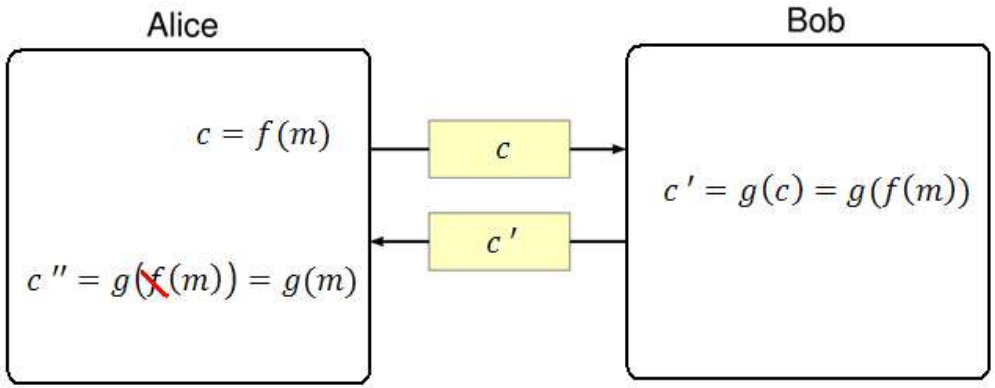
- Directive (EU) regarding electronic signature (1999)
- The law of electronic signature in RO (2001)
- The law of the time stamp in RO (2004, 2008)
- The law of electronic archiving in RO (2007)

- ***Regulation*** **(EU) No 910/2014 – eIDAS**
  - Next version eIDAS-v2 (2023 ?!)

Blind signatures

- Introduced by David Chaum (1983)
- Digital signature in which the content of a message is hidden (blinded) before it is signed
- Applications: e-cash, e-voting protocols

- Protocol definition:
  - Alice sends a piece of information derived from message m to Bob
  - Bob signs and returns the signature to Alice
  - From this signature, Alice can compute Bob's signature on a message m of Alice's choice.
  - At the completion of the protocol, Bob knows neither m, nor the signature associated with it.



[wikipedia.org]

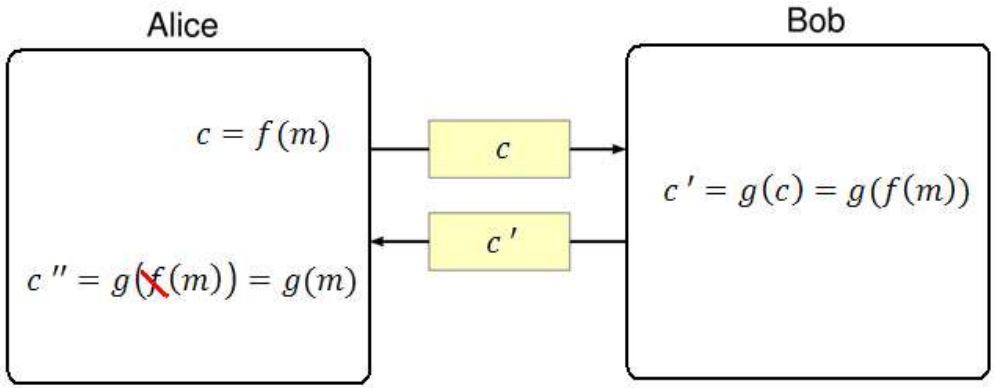
Blind signatures – *Chaum Blind Scheme*

Actors: sender Alice; signer Bob

- Bob's RSA public and private key are as usual
- Let k a random secret integer chosen by Alice, satisfying $0 < k < n$

Protocol actions:

- (blinding process) Alice computes $m^* = m \cdot k^e \pmod n$, then sends m^* to Bob
  - Note:$(m \cdot k^e)^d = m^d \cdot k$
- (signing process) Bob computes $s^* = (m^*)^d \pmod n$, then sends s^* to Alice
- (unblinding process) Alice computes $s = k^{-1} \cdot s^* \pmod n = m^d \pmod n$



[wikipedia.org]

Blind signatures – *Chaum Blind Scheme*

Actors: sender Alice; signer Bob

- Bob's RSA public and private key are as usual
- Let k a random secret integer chosen by Alice, satisfying $0 < k < n$

Protocol actions:

- (blinding process) Alice computes $m^* = m \cdot k^e \ (mod\ n)$, then sends m^* to Bob
  - Note:$(m \cdot k^e)^d = m^d \cdot k$
- (signing process) Bob computes $s^* = (m^*)^d \ (mod\ n)$, then sends s^* to Alice
- (unblinding process) Alice computes $s = k^{-1} \cdot s^* \ (mod\ n) = m^d \ (mod\ n)$

Dangers of RSA blind signing

- It is possible to be tricked into decrypting a message by blind signing another message
- An attacker can provide a blinded version of a message m encrypted with the signer's public key, m^* for them to sign

- Due to this, the same key should never be used for both encryption and signing purposes

Electronic Signatures

Bibliography:

D. R. Stinson,
Cryptography: Theory and Practice

B. Schneier,
APPLIED CRYPTOGRAPHY

W. Stallings,
Cryptography and Network Security